VECTOR

Flash Bootloader Hardware

TechnicalReference

FBL TriCore MultiCan

Version 2.6

Authors	visach, vaddod, visjhg; visrr, visrie, visrcn
Status	Released

Document Information

History

Author	Date	Version	Remarks
visach	2013-01-21	2.0	Update to new document template, added new derivative group Aurix
visach	2013-10-21	2.1	Update CAN settings and copy code to RAM on startup description
vaddod, visach	2014-03-13	2.2	Small clarification for timer configuration, Copy code to RAM on startup description clarified
visach	2015-07-14	2.3	Handling of WOP blocks (Aurix derivatives) Configuration of flash bank sizes
visjhg, visach	2016-02-11	2.4	Handling of internal watchdog (Aurix derivatives), hardware specific callback functions added
visrr, visrie	2020-07-23	2.5	Added TC3xx protection information (chapter 3.1.3) Added HIS extension API (chapter 5.1) Added information regarding swap (chapter 2.2.4)
visrcn	2020-10-14	2.6	Removed information for the SwapActivate function (see ESCAN00101898)

Table 1-1 History of the Document

Reference Documents

No.	Source	Title	Version
[1]	Infineon	User's manuals (various derivatives)	n.a.
[2]	Vector	MICROSAR Fr ERAY – Communication Controller E-Ray Tricore	1.37
[3]	Vector	MICROSAR CAN Driver – Infineon Tricore MultiCAN	1.25.00
[4]	Infineon	User Manual Tc3xx	1.2.0
[5]	HIS	Functional Specification of a Flash Driver	1.3

Table 1-2 References Documents

Contents

1 Introduction 7
1.1 Configurations covered by this Technical Reference 7
1.1.1 Microcontrollers 7
1.1.2 Compilers 7

2 Memory Model 8
2.1 [#hw_mem] - Design the Memory Layout 8
2.2 Memory Mapping 8
2.2.1 Memory mapping of application and FBL 8
2.2.2 Memory range of the FBL 8
2.2.3 Memory range of the application 9
2.2.4 Swap Partitions 10
2.3 [#hw_intvect] - The Interrupt Vector Tables 10
2.3.1 Bootloader Vector Table 10
2.3.1.1 Tasking compiler 10
2.3.1.2 HighTec/GNU compiler 10
2.3.2 Application Vector Table 10
2.3.3 Vector base address register 11
2.3.4 Special measures for NMI 11

3 Hardware Layer 12
3.1 Flash Memory and Flash Driver 12
3.1.1 Flash Memory Layout 12
3.1.1.1 AUDO family derivatives 12
3.1.1.2 Aurix family derivatives 12
3.1.2 ECC protection 14
3.1.2.1 AUDO family 14
3.1.2.2 AURIX family 14
3.1.3 Enabling and Disabling protection 14
3.1.3.1 Aurix TC3xx 14
3.1.3.1.1 Global Bootloader unprotection 15
3.1.3.1.2 Use case specific unprotection 15
3.1.3.1.3 Long running customer code 17
3.1.4 Supported flash masks 18
3.1.5 Flash driver configuration 18
3.1.5.1 AUDO family 18
3.1.5.2 Aurix family 18
3.1.6 Rebuilding the flash driver binary 20
3.1.7 [#hw_size] - Flash Segment Size 21

© 2020 Vector Informatik GmbH

3.1.8 Read ECC safe ... 21
3.1.8.1 Read without trap generation ... 21
3.1.8.2 Implementation of a user defined read function ... 22
3.2 CAN Driver ... 22
3.2.1 CAN Nodes ... 22
3.3 FlexRay Driver ... 23
3.4 Timer ... 23
3.5 Controller Startup Behavior ... 24
3.5.1 AUDO family ... 24
3.5.2 Aurix family ... 24
3.5.2.1 BMI ... 24
3.5.2.2 Multi Core ... 25
3.6 Startup Code ... 25
3.7 Watchdog ... 26
3.7.1 AUDO family ... 26
3.7.2 Aurix familiy ... 26
3.8 Hardware specific callback functions ... 28
3.8.1 Hardware lock/unlock callbacks ... 28
3.8.2 CAN initialization delay ... 29

4 Build ... 30
4.1 Code executed from RAM ... 30
4.1.1 Use of relocatable code ... 30
4.1.2 Use of code linked to RAM ... 30
4.1.2.1 Tasking VX compiler ... 31
4.1.2.2 HighTec/GNU compiler ... 31

5 API Description ... 32
5.1 Flash Driver (TC3xx) ... 32
5.1.1 FlashDriver_UcbEraseSync ... 32
5.1.2 FlashDriver_UcbWriteSync ... 32
5.1.3 FlashDriver_UcbWriteConfirmed ... 33
5.1.4 FlashDriver_UcbReadSync ... 33
5.1.5 FlashDriver_UcbDisableProtection ... 34
5.1.6 FlashDriver_ResumeProtection ... 34
5.1.7 FlashDriver_DisableProtection ... 35
5.1.8 FlashDriver_ReplaceLogicalSector ... 35
5.1.9 FlashDriver_GetReplaceResult ... 36
5.1.10 FlashDriver_SwapPartitions ... 36

6 Glossary and Abbreviations ... 38

© 2020 Vector Informatik GmbH

6.1 Abbreviations 38

7 Contact 39

© 2020 Vector Informatik GmbH

Illustrations

Figure 1-1 Manuals and References for the Flash Bootloader 7
Figure 2-1 Memory Mapping 8
Figure 3-1 TriCore Aurix specific configuration 19
Figure 3-2 Configuration of WOP block memory device 20
Figure 3-3 Configuration of flash blocks as WOP memory device 20
Figure 3-4 Flash Driver Directory 20
Figure 3-5 CAN node configuration 23
Figure 3-6 CAN Module Register Map 23
Figure 3-7 GENy timer settings 24

Tables

Table 1-1 History of the Document 2
Table 1-2 References Documents 2
Table 3-1 Supported flash masks 18
Table 3-2 Flash Segment Size 21
Table 3-3 Timer Configuration Code 24
Table 3-4 Appl_UnlockInit 28
Table 3-5 Appl_LockInit 28
Table 3-6 ApplCanWaitSetMCR 29
Table 5-1 API Description: FlashDriver_UcbEraseSync 32
Table 5-2 API Description: FlashDriver_UcbWriteSync 33
Table 5-3 API Description: FlashDriver_UcbWriteConfirmed 33
Table 5-4 API Description: FlashDriver_UcbReadSync 34
Table 5-5 API Description: FlashDriver_UcbDisableProtection 34
Table 5-6 API Description: FlashDriver_ResumeProtection 35
Table 5-7 API Description: FlashDriver_DisableProtection 35
Table 5-8 API Description: FlashDriver_ReplaceLogicalSector 36
Table 5-9 API Description: FlashDriver_GetReplaceResult 36
Table 5-10 API Description: FlashDriver_SwapPartitions 37

© 2020 Vector Informatik GmbH

1 Introduction

This document covers the hardware-related particularities of the Flash Bootloader. It complements the explanations started in the user manual with hardware-specific details. All references there are resumed here in this document again and explained in detail.

The connection between a reference in the user manual and its specific description in this document is the headline. Both the reference and its explanation can be found below the same headline.

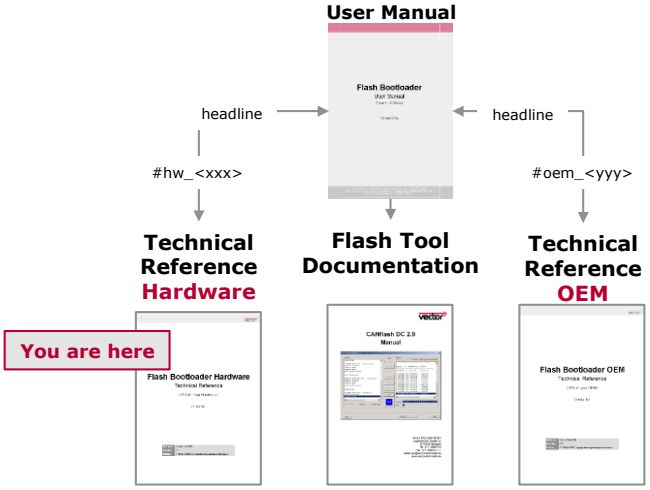


Figure 1-1 Manuals and References for the Flash Bootloader

Additionally this headline is marked with the ID of the reference from the User Manual. This ID looks like: **[#hw_<xxx>]**.

1.1 Configurations covered by this Technical Reference

This Technical Reference covers the following controllers and compilers:

1.1.1 Microcontrollers

- AUDO NextGeneration (e.g. TC1766 and TC1796)
- AUDO Future (e.g. TC1767 and 1797)
- AUDO MAX (e.g. TC1798, TC1782, TC1784, TC1724)
- AURIX (e.g. TC233, TC264, TC275, TC297)

1.1.2 Compilers

- Tasking VX compiler
- Hightec GNU compiler

2 Memory Model

2.1 [#hw_mem] - Design the Memory Layout

For more general information about this see the UserManual_FlashBootloader in the chapter **Design Memory Layout**.

2.2 Memory Mapping

2.2.1 Memory mapping of application and FBL

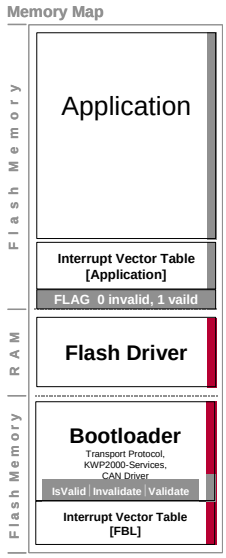


Figure 2-1 Memory Mapping

2.2.2 Memory range of the FBL

The Bootloader has to be placed in a flash block containing the reset vector (0xA0000000) if an AUDO device is used or at an address containing a valid BMHD structure. Please see chapter 3.5 - Controller Startup Behavior for more information.

The reset vector or BMHD and the application vector table have to be placed in uncached memory. All other parts of the Bootloader can be safely placed into cached memory address space.

If the Bootloader occupies more than one flash block, every additional flash block on the device can be chosen. Please make sure that the flash block is configured as “Protected” so that it cannot be erased or overwritten and consider the erase/write limitations on AUDO devices (chapter 3.1.1 - Flash Memory Layout).

2.2.3 Memory range of the application

The application can be placed in the first flash sector after the Bootloader and may occupy all memory regions which are not reserved for the Bootloader. Please make sure that all flash blocks occupied by the Bootloader are configured to “Protected” in the flash block table.

Please note that there are a few recommendations how to place the application software on TriCore controllers:

- It is recommended to place the application area (code and data) into the uncached memory area (0xA0000000).
- Some TriCore derivatives[1] include so-called logical blocks. These flash blocks have a reduced count of erase cycles. It is recommended to combine all logical blocks of one physical block into either Bootloader area or application area.

Flash memory is shown in two address ranges on TriCore controllers. The address range 0x8… is accessed via cache, the address range 0xA… is accessed without cache, but both address ranges use the same physical flash memory. The following problems occur with this memory setup:

- The addresses used in code determine, if data or code is accessed cached or un-cached.
- A resulting download file normally includes cached and un-cached parts.
- Normally, a download sequence cannot handle downloads which are split into 2 address areas but occupy the same physical memory.

To handle these problems, the download file has to be merged into one address range (cached or un-cached).



Note
It is recommended to execute the remapping process with a tool which could change Intel Hex or S-Record files before downloading, but after linking the file.

If the output file is used, the behavior of the downloaded data isn’t changed whereas changing addresses in the linker control file changes the behavior.



Note
It is recommended to remap the data into the un-cached memory area. Flashing into the cached memory area is not possible and the flash driver itself doesn’t have to remap the data to un-cached addresses if they are already mapped to un-cached addresses.

[1] AUDO derivatives

2.2.4 Swap Partitions

The swap partitions in the Tricore is handled through the programming of the appropriate UCB blocks. This configuration is reset-persistent, meaning that after reset or switch off-switch on, the swap configuration will be preserved. Furthermore, the swap takes effect after a reset has been performed.

2.3 [#hw_intvect] - The Interrupt Vector Tables

For more general information about this see the UserManual_FlashBootloader in the chapter The Interrupt Vector Tables and Fbl_vect.c / Applvect.c(.h) - The Interrupt Vector Tables.

2.3.1 Bootloader Vector Table

TriCore controllers do not require an interrupt redirection table because the interrupt vector base register (IVBR) can be configured. However, depending on the compiler, a vector table may be needed to configure the trap handlers.

2.3.1.1 Tasking compiler

Trap handlers are configured by the compiler and linker if the Tasking tool suite is used. No fbl_vect.c module is included in the Bootloader.

2.3.1.2 HighTec/GNU compiler

A trap table is needed if the HighTec/GNU compiler is used. This trap table is located in the fbl_vect.c module provided with the Bootloader delivery.

2.3.2 Application Vector Table



Note
The fbl_applvect.c file doesn't contain the application vector table itself, but some important information which is used by the Bootloader.

The application vector table provided by the Bootloader contains some additional information needed by the Bootloader to start the application:

- A flag that tells the Bootloader that the application interrupt vector table is valid/present. This flag must be set to APPLVECT_FROM_APPL in the application.
- The start address of the application.



Caution
This table must be located at a fixed address to provide the Bootloader with the memory address of the application interrupt vector table validation flag. Please make sure to locate the application interrupt vector table validation flag at the same address in the Bootloader and in the application itself.

It is also recommended to place the application vector table in a non-cached memory area.



Example
An example implementation is provided with every Bootloader delivery:

- The Bootloader demonstration project includes a placeholder called fbl_applvect.c
- The demonstration application includes the file applvect.c. This file includes the address of the application's startup code which is used to start the application software by the Bootloader.

2.3.3 Vector base address register

TriCore controllers include base registers to relocate traps (BTV, base trap register) and interrupts (BIV, base interrupt register) to the application memory space. Because of this, there is no interrupt vector redirection table included in the Bootloader.

2.3.4 Special measures for NMI

Because of a hardware issue of the internal watchdog in AUDO devices, a second watchdog initiated reset may incorrectly result in a permanent reset lock. The ECU can only be reactivated by a hardware reset or power on reset.

To avoid this situation, an additional hardware specific callback exists. It can be found in the file fbl_apwd.c. ApplFblWDNmiTrap() acts as a trap handler for the non maskable interrupt (trap class 7) generated by the watchdog shortly before the reset is executed. Inside the trap handler a software reset is executed which preempts the watchdog reset.

To enable the trap handler the compiler has to be informed about the user specific handler. Additionally the activation of the non maskable interrupt is necessary during Bootloader initialization.

In case an external watchdog is used the trap handler isn't necessary and can be removed completely. The trap handler is already removed on Aurix devices.

3 Hardware Layer

3.1 Flash Memory and Flash Driver

3.1.1 Flash Memory Layout

The flash memory is divided into flash blocks. The size of the flash blocks differs from derivative to derivative. Please see the hardware manual of your derivative for a detailed description of its flash memory layout.

3.1.1.1 AUDO family derivatives

Caution
On AUDO devices, the flash block table configured in GENy is used by the flash driver. It has to be configured to the exact memory layout described in the hardware manual. Otherwise, erase and write accesses will fail.

AUDO devices include 2 types of flash blocks: Logical and physical sectors. These flash block types use different erase algorithms – logical sectors are specified for less erase cycles.

Caution
Infineon specifies 50 or 100 program/erase cycles for a logical sector. After the erase cycle count reaches the specified value, the sector cannot be erased any more.

Logical sectors can be refreshed by erasing the corresponding physical sector. This erases all logical sectors located in the physical sector. Please note that there are 2 options to deal with this restriction:

- Reserve an entire physical sector for the Bootloader. The application memory layout should be configured to include complete physical sectors into one logical block or erase area.
- Use the flash attempt counter to make sure only 50 or 100 program/erase cycles are allowed for the corresponding logical block.

3.1.1.2 Aurix family derivatives

Aurix family derivatives also organize flash blocks in logical and physical sectors. This organization is handled internally by the flash driver.

To optimize erase times, all logical sectors of one physical sector can be erased with one erase command. To achieve this behavior, you have to combine logical sectors of one or more physical sectors and flash modules into one flash block in GENy because most Bootloader diagnostic modules will erase every flash block separately.

© 2020 Vector Informatik GmbH



Caution
If you combine several flash blocks into one flash block, double check that no Bootloader memory is included into this flash block.

Please verify that the combined flash block is aligned to the start address of one logical sector and the end address of another logical sector.

Program flash sectors are endinit protected on system level. If access to program flash write commands is enabled, the watchdog module supervises the unlock times.

If the unlock times exceed a timeout, a watchdog reset is issued. Code parts which have to be executed with unlocked endinit protection are kept as short as possible, but a small timeout may lead to problems.



Caution
Please check carefully that flashing works with your system watchdog timeout configuration.

Flash blocks on Aurix controllers can be configured to be writeable only once (so-called write-page once flash blocks). These flash blocks can be configured in the device's UCB flash area and have to be written with special flash commands. Please see the corresponding Infineon User Manual for handling information.

The flash driver delivered with Aurix devices includes a special driver variant to write these flash blocks.



Note
If a WOP flash block is configured to be written via a UDS download, please keep in mind that this block cannot be erased and the flash attempt counter has to be limited to one flash attempt.



Caution
Errors in UCB contents can lock the startup sequence of TriCore controllers. Please be careful when changing the UCB blocks.

© 2020 Vector Informatik GmbH

3.1.2 ECC protection

The internal flash memory of TriCore devices can be protected by ECC checksums. The checksum type and handling differs depending on the chip families:

3.1.2.1 AUDO family

ECC handling is deactivated by default on these controllers. No special precautions are implemented to handle ECC failures on these devices.

3.1.2.2 AURIX family

ECC handling is activated by default on Aurix devices. There are two types of ECC checksums available:

- Safety ECC: Default configuration, includes memory address into checksum. If this configuration is enabled, erased flash areas contain ECC errors.
- Legacy ECC: Has to be enabled manually, no address information is included into checksums.



Caution
The flash write functions use the selected algorithm to program the checksums. The Bootloader configuration must match the application configuration – otherwise checksums for a different algorithm than the used read algorithm are programmed.

If there are no special requirements, we propose to use the standard algorithm (Safety ECC).

ECC errors are reported by a SFR. No trap is generated by default.



Caution
If the ECC handling traps are activated, endless resets or traps can be caused by flash accesses to areas which have not been checked before, e.g. presence patterns. If you activate ECC handling traps in Bootloader context, you have to use a read function which can handle possible traps.

Two algorithms can be used to read from ECC protected areas:

- Standard algorithm with deactivated traps.
- User defined algorithm with trap handling.

Please see chapter 3.1.8 - Read ECC safe for more information.

3.1.3 Enabling and Disabling protection

3.1.3.1 Aurix TC3xx

TC3xx family supports a disable and resume protection sequence (see [4] User Manual Tc3xx). Flash driver directly support these functions with following APIs:

`FlashDriver_DisableProtection`

`FlashDriver_ResumeProtection`



Caution
Flash driver will not call disable or resume protection by itself nor call any user callouts with the purpose of enabling or disabling protection.
If passwords are used user must disable protection before flash driver will be called.



Note
Bootloader or flash driver will not set any passwords. So, there are no “default” passwords, which needs to be known or changed by configuration.



Note
System Reset will automatically re-enabling the protection. Due to the fact that Bootloader will always be left by Reset resuming protection is optional.

Because the Bootloader will be called to modify flash only in a safe state, Vector recommends simple protection use cases.

3.1.3.1.1 Global Bootloader unprotection

Before first call of memory initialization and after memory deinitialization the APIs for setting and resetting Flash Programming Voltage:

`ApplFblSetVfp`

`ApplFblResetVfp`

can be used disabling and enabling protections for all possible use cases. This is the easiest way to get everything running.

3.1.3.1.2 Use case specific unprotection

You might have special independent use cases for the flash driver like

- Application PFLASH Update through Bootloader
- Bootloader PFLASH Update through Bootloader Updater
- OTA use case: Execute only SWAP and reset
- …

Note
Bootloader will only call standard HIS API (init, deinit, erase write, read) in core files. All extended APIs must be called in user callouts. All flash driver APIs are synchronous. Therefore, flash driver extended API functions can easily surrounded by protection API.

Note
You can use differences in known sequences to identify the use case and set unprotection areas accordingly. I.e. SWAP sequence for OTA use case and Bootloader Software Update sequence may both call SetVfp and ResetVfp API. So these APIs cannot be used without further flags to identify the use case.

But Bootloader Software Update may use a download able driver, which needs to be verified or activated in special callouts and it make more sense to place unprotection call for PFLASH there.

Example
OTA use case: Swap Partitions (API: `FlashDriver_SwapPartitions`)

Look for API call and surround it by protection API:

```
vuint32 password[8] = {0xAAuL, 0xAAuL, 0xAAuL, 0xAAuL, 0xAAuL, 0xAAuL, 0xAAuL, 0xAAuL};
if( IO_E_OK != FlashDriver_DisableProtection(0x0017,password))
{
  result = FlashDriver_SwapPartitions();
  FlashDriver_ResumeProtection();
}
```

Example
Bootloader vs. Bootloader Updater:

Both projects use different sources. It shall be easy to find unique APIs to disable different PFLASH areas



Example
OTA use case PFLASH: Bank A or B?

In OTA use case the Bootloader work either on A or B bank. The key for understanding how it works is the API:

`MemDriver_SetAddressArea`

`MemDriver_GetAddressArea`

Please check where the setter API will be called to understand where the Bootloader selects the bank. Now you can use the getter API in any following callout to limit the unprotection to a limited area.

3.1.3.1.3 Long running customer code

The Bootloader and its API are mostly synchronous with a polling approach. If you have an asynchronous API for unlocking, you can look for different user callouts to initialize unlocking and later set a guard to ensure that it has finished. This is for several reasons quite tricky and therefore we recommend having one long running sequence respecting Bootloader handling.

Bootloader based on polling approach around FblLookForWatchdog(). Please have an eye on the following APIs:

`FblLookForWatchdog`

`FblLookForWatchdogVoid`

`FblRealTimeSupport`

The first 2 maintain the watchdog and base timing and even send Response Pending if initial Response Pending was already sent. These functions can be used for any long operation in any callout.

In case we are in a callout in the middle of diagnostic download, it may make more sense to call the 3rd function. This function respects P2 and P2* timing and sends automatically Response Pending in the correct timing.



Example

```
UserAsync_StartMyFunction()
while(!UserAsync_IsMyFunctionFinished())
{
  /* outside download sequence or inside knowing RCRRP was already sent */
  FblLookForWatchdogVoid();
  /* inside UDS sequence */
  (void)FblRealTimeSupport();
}
/* continue with next step */
```

3.1.4 Supported flash masks

The flash drivers for TriCore have been tested with certain mask versions. Please note that this list is not complete.

Derivative	Flash masks supported beginning with
TC1766	BB
TC1767	AC
TC1796	BC
TC1797	AA
TC275	CA ES
TC297	AA EES

Table 3-1 Supported flash masks

3.1.5 Flash driver configuration

3.1.5.1 AUDO family

Audo family flash drivers don’t need any special configuration options.

3.1.5.2 Aurix family

Aurix family flash drivers include some special configuration options:

- Read erased memory areas: Aurix flash drivers include a workaround to read deleted flash areas and to check if the memory at some address is deleted (including the ECC bits). This workaround performs the following sequence:
  1. Read memory
  2. If an ECC error occurred, deactivate the error correction

3. Read memory again. If all flash cells contain erased memory, return a special value. Otherwise return a simple error
4. Reactivate the error correction.

This feature can be configured in GENy:

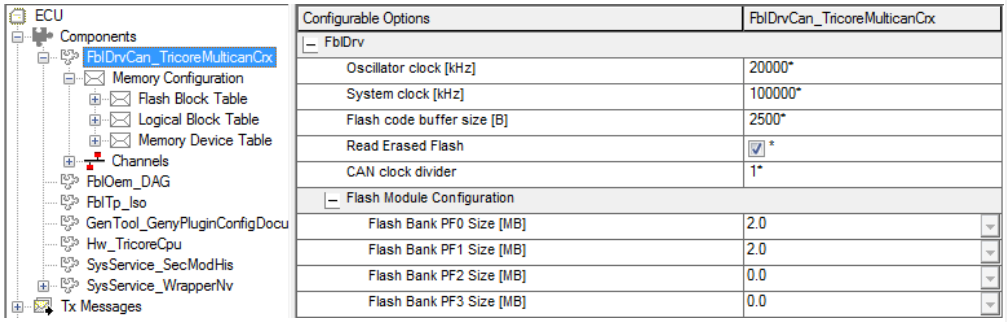


Figure 3-1 TriCore Aurix specific configuration

- The TriCore GENy module includes a flash bank configuration. Please check the flash setup of the derivative you are using and adapt the flash module configuration section shown in Figure 3-1.
- Support erase sectors of 8 KBytes and 64 KBytes in data flash. The data flash memory of Aurix devices should support erase sectors of 8 Kbytes size, but early versions (µCode versions 0/1) support erase sectors of 64 KBytes. Support of the 64KB sectors can be enabled with a user configuration file:

Example
Enable 64KB erase blocks in data flash:

```
/* Enable flash uCode version 0 - 64KB Blocks in data flash */
#define FLASH_ENABLE_UCODE_V0
```

Writing of WOP blocks can be configured via GENy. They can be either configured in the flash block table/ logical block table or a user configuration file can be used to activate WOP block support.

Example
Enable WOP block support via user configuration file

```
/* Enable flashing of WOP flash blocks */
#define FLASH_ENABLE_WOP_SUPPORT
```

This switch activates a set of flash driver functions (FlashWopDriver_*). Please use these functions for blocks configured as WOP blocks only. The flash driver (flashdrv.c) doesn't have to be recompiled if WOP support is used.

If WOP blocks should be used as a download target, the WOP flash blocks have to be configured in GENy:

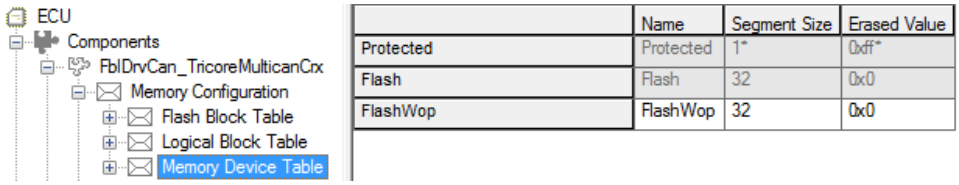


Figure 3-2 Configuration of WOP block memory device

	Start Address	End Address	Memory Device	Description	Logical Block
0xA0200000	0xa0200000	0xa0203fff	FlashWop	PF1 S0: 16K	OtpTest
0xA0204000	0xa0204000	0xa0207fff	FlashWop	PF1 S1: 16K	*
0xA0208000	0xa0208000	0xa020bfff	FlashWop	PF1 S2: 16K	*

Figure 3-3 Configuration of flash blocks as WOP memory device

3.1.6 Rebuilding the flash driver binary

TriCore flash drivers are non-relocatable. They have to be linked to their execution address.

Please use the supplied build environment to rebuild the flash driver. Please use the same compiler settings which are used in the make file. Other settings have not been tested and are therefore not allowed.

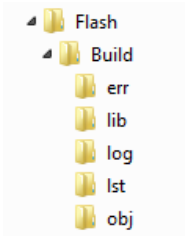


Figure 3-4 Flash Driver Directory

Flash drivers are delivered in a directory like the directory shown in Figure 3-4. Depending on the delivery structure, the base directory can be named "DrvFlash" instead of "Flash".

The subdirectory "Build" contains a build system to rebuild the flash driver. Compared to the Bootloader demonstration build system, it additionally contains a stub of the derivative's linker definition whereas the Bootloader demo includes the compiler's default files. This stub is needed to link the flash driver directly into RAM.

Caution
TriCore controllers include RAM which can hold executable data and RAM which can only hold data. The flash driver has to be linked into a RAM area which can hold program code, e.g. PMI Scratch Pad RAM.

3.1.7 [#hw_size] - Flash Segment Size

For more general information about this see the UserManual_FlashBootloader in the chapter **Flash Segment Size**

The flash segment size describes the minimum writeable data size:

Flash memory	Flash segment size
AUDO devices, program flash	256 Bytes
AUDO devices, data flash	128 Bytes
AURIX devices, program flash	32 Bytes
AURIX devices, data flash	8 Bytes

Table 3-2 Flash Segment Size

3.1.8 Read ECC safe

Note
This chapter currently describes Aurix devices only.

3.1.8.1 Read without trap generation

This function is implemented by default. No special settings have to be made, but trap generation must not be enabled by the user during the devices startup.

If trap generation is needed within the application software, it should be enabled during the application's startup phase.

The read function reports all uncorrectable errors, but ignores corrected bits.

3.1.8.2 Implementation of a user defined read function

If the included read function cannot be used with your configuration, the read function can be deactivated and substituted by a user specific implementation.



Caution
The Bootloader uses the read function for all flash read accesses. Please verify that the function implements the same external behavior as the original function.

The user read function feature cannot be configured by GENy, it has to be activated with a user pre-configuration file.



Example
User pre-configuration file (include into GENy):

```
#define FBL_ENABLE_USR_RREAD_SYNC
```

The read function has to use the following interface:



Example
User defined read function:

```
IO_ErrorType V_API_NEAR FlashDriver_RReadSync(
IO_MemPtrType, IO_SizeType, IO_PositionType );
```

The read function has to fulfill a few requirements while executed in the Bootloader's context:

- The read function has to return IO_E_OK (0x00u, no error detected) or IO_E_NOT_OK (0x01u, read error detected), read data is stored in a buffer (1st parameter with type IO_MemPtrType).
- During read, FblLookForWatchdog() has to be called at least once a millisecond.

3.2 CAN Driver

The Bootloader currently supports TriCore controllers with MultiCAN CAN cells.

3.2.1 CAN Nodes

The TriCore controller has 2 to 4 CAN nodes depending on the derivative[2].

[2] TC1766: 2 CAN nodes, TC1767: 2-3 CAN nodes depending on the package, TC1796/TC1797: 4 CAN nodes.

The CAN node can be selected using GENy (“Physical Node” configuration field, red). The CAN port pins can be selected using the configuration field “IO Port, green”. This value is used as RXSEL value in the NPCR (Node port control) register.

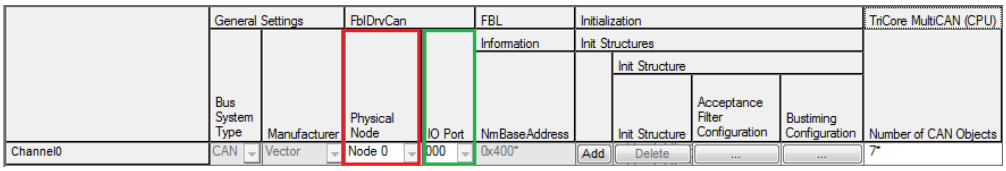


Figure 3-5 CAN node configuration

The CAN register structure is configurable using GENy as well. By default, this setting is not changeable but preconfigured to the derivative which has been specified in the Bootloader order. If these settings have to be changed, they can be enabled on special request.

MultiCAN Module Register Map	
Number of CAN Nodes	4*
Total number of CAN Nodes	14*
Number of List Registers	8*
Total number of List Registers	16*
Number of Message Pending Registers	8*
Total number of Message Pending Registers	16*
Number of Message Index Registers	8*
Total number of Message Index Registers	16*
Number of Message Objects	256*

Figure 3-6 CAN Module Register Map

3.3 FlexRay Driver

FlexRay Bootloaders are delivered with Microsar FlexRay components. There is separate documentation [2] provided with the FlexRay components which shows how to configure the FlexRay component.

Generic, Bootloader specific FlexRay settings are described in the OEM’s Bootloader Technical Reference.

3.4 Timer

The hardware independent modules of the Bootloader require a 1ms time base to derive the necessary call cycles of cyclic functions from that time base.

Like the other hardware specific configurations, the timer module is also configured with the configuration tool GENy. Please specify the frequency connected to the timer module in GENy (setting “System clock [kHz]”).

© 2020 Vector Informatik GmbH

Configurable Options	FblDrvCan_TriCore
FblDrvCan	
Oscillator clock [kHz]	20000*
System clock [kHz]	100000
Watchdog function size [B]	512*
Flash code buffer size [B]	2500*
Read Erased Flash	☑ *
CAN clock divider	1*

Figure 3-7 GENy timer settings

Symbolic constant	Description
FBL_COMPARE_REG_VAL	Timer reload value for 1ms base cycle.

Table 3-3 Timer Configuration Code

The Bootloader uses the System Timer 0 module (STM_TIM0). No special flash programming timing is required by the flash driver.

Caution
The GENy configuration option for “General Timer Algorithm -> Timer Clock [kHz]” may be displayed but is not used for the TriCore.

3.5 Controller Startup Behavior

3.5.1 AUDO family

TriCore derivatives of the AUDO device family include one processor core. This processor core is used by the Bootloader.

The default configuration provided with the Bootloader demo assumes that the software is started from internal flash. If a different startup code is selected (e.g. Alternate Boot Mode), the necessary code has to be added to the Bootloader.

3.5.2 Aurix family

3.5.2.1 BMI

Aurix devices are always started with a BMI data structure. This data structure is called BMHD and has to be programmed into the ECU with the Bootloader.



Caution
If no valid BMHD is programmed into the ECU, your Bootloader won't start and debugger access to the controller remains locked. Take special care not to erase all BMHD data structures at one time.

The BMHD has to be placed to specific addresses (e.g. 0xA0000000) and therefore the Bootloader has to occupy a flash block with a valid BMHD position.



Caution
If some other software code or data block is placed at a BMHD address which is evaluated before the Bootloader's BMHD is evaluated, this software or data will be started if a valid BMHD structure is recognized. If you don't place the Bootloader in the flash block with the first BMHD position, you have to make sure that the positions evaluated before the Bootloader's BMHD never contain a valid BMHD structure.

3.5.2.2 Multi Core

Aurix derivatives (TC27x) include up to 3 processor cores. The Bootloader always uses processor core 0, all other processor cores remain stopped.



Note
If any other core than processor core 0 shall be used, the application software has to activate these processor cores during the application software startup sequence.

Flash memory is shared between all processor cores, but each processor core contains local RAM. To improve execution speed of the Bootloader, we recommend to link both data and RAM executable code to the local RAM of CPU0.

3.6 Startup Code

There is a sample startup code provided with the Bootloader demonstration project. This startup code performs several configuration and preparation tasks before the Bootloader's main function can be started.



Caution
The startup code is provided as a part of the Bootloader demonstration project. Please use your own startup code or review the startup code carefully to ensure the startup code is correct and fits into your use case.

The following tasks have to be handled by the startup code:

- NMI initialization.
- RAM initialization.
- Stack and data base pointer initialization.
- Copy of linked to RAM routines from flash to RAM.



Caution
Keep in mind that there will be two startup codes executed subsequently, first the startup code of the Bootloader, then the startup code of your application.

Be careful with registers that can be written only once after reset.

3.7 Watchdog

The demo Bootloader contains an exemplary implementation to serve the internal watchdog.

The watchdog trigger is closely related to the register unlock sequence. But the handling slightly differs, depending on the controller family (AUDO or Aurix).

3.7.1 AUDO family

AUDO devices only have a single watchdog instance and although the used password is user-definable, a single sequence without any control-flow decisions can be used.

Because of this the reference implementation in the user callbacks ApplFblInit and ApplFblWdTrigger contain their own copy of the trigger sequence. Hence no additional requirements have to be fulfilled.

3.7.2 Aurix familiy

Aurix devices have dedicated watchdogs for each CPU core, as well as a global safety watchdog instance. Additionally the watchdogs can be configured to use an automatic password sequencing scheme, instead of a static password.

This complicates the implementation and makes code duplication in the user callbacks ApplFblInit and AblFblWdTrigger impractical. Instead the functions to modify the watchdog parameters and trigger the watchdog, using the register lock function, are exported by the hardware layer.



Caution
The watchdog trigger will only work reliably during flash operations, when all necessary functions and constants are linked to RAM.

Besides FblLookForWatchdog and ApplFblWdTrigger, the following functions and constants, found in fbl_hw.c, have to be linked to RAM:

- FblHwRegisterLock
- FblHwRegisterGetIndex
- FblHwRegisterPasswordAccess
- FblHwRegisterModifyAccess
- fblWdtRegisters

See chapter 4.1.2 for information on how to link code to RAM.

3.8 Hardware specific callback functions

TriCore Bootloaders can include hardware specific callback functions which are called by the communication driver. Please see [2], [3] or a similar document provided with your delivery for details.



Note
Hardware specific callback functions are available if CANbedded or Microsar communication drivers are used in the Bootloader package.

3.8.1 Hardware lock/unlock callbacks

These functions are used to allow write accesses to the clock control register of the communication module.



Caution
Depending on the communication stack, the names of the following functions can differ slightly. Please adapt the function names in this case.

Prototype
`void Appl_UnlockInit(void)`
Functional Description
Register access unlock request from communication stack initialization function. The register write access to the clock configuration register of the communication stack has to be allowed here.
Particularities and Limitations
> If hardware access on system level is activated, the time until the lock sequence has to be called is limited.
Call context
> Communication driver initialization

Table 3-4 Appl_UnlockInit

Prototype
`void Appl_LockInit(void)`
Functional Description
Register access lock request from communication stack initialization function. The register write access to the clock configuration register of the communication stack has to be locked here.
Particularities and Limitations
> Has to be called before the allowed unlock time expires.
Call context
> Communication driver initialization

Table 3-5 Appl_LockInit

3.8.2 CAN initialization delay

Prototype
`void ApplCanWaitSetMCR(void)`
Functional Description
Configurable delay after the CAN cell MCR was set.
Particularities and Limitations
> Required delay has to be set according to the required delay documented in [1].

Table 3-6 ApplCanWaitSetMCR

4 Build

4.1 Code executed from RAM

The Bootloader has to execute some code from RAM while flash memory is erased or written. Depending on the compiler used to compile the Bootloader, one of the two available approaches is used (relocatable code or code linked to RAM). Code linked to RAM is used by default, but use of relocatable code is possible on AUDO derivatives with HighTec/GNU compilers.

4.1.1 Use of relocatable code

Some configurations which use the HighTec/GNU compiler on AUDO derivatives can use relocatable code for watchdog handling. The watchdog handling functionality is copied by the Bootloader itself during the startup phase. The two functions FblLookForWatchdog() and ApplFblWdTrigger() are copied to RAM in this configuration. No special precautions have to be taken while rebuilding the Bootloader.



Caution
No function calls from ApplFblWdTrigger() are allowed if relocatable code is used for watchdog handling because function calls are normally not relocatable.

4.1.2 Use of code linked to RAM

Code is linked to RAM using special compiler/linker features. Basically, all compilers use the same method:

- Functions which have to be linked to RAM are placed in special sections or complete modules are marked to be linked to RAM.
- The compiler links these modules to RAM address space.
- An image of the code and data linked to RAM is created. This image is stored into flash memory.
- Some address information to find the image in flash and target addresses in RAM is created.

Code linked to RAM using this method can be debugged normally because the debug symbols are located at the execution addresses of the code.



Note
The addresses which can be supplied to some jump instructions are not big enough to perform a jump from flash memory to RAM and back. Bootloaders which have to execute such jumps either include a memmap.h file which inserts far jumps or have to be compiled with far jump compiler options.

The implementations of code in RAM differ for every compiler.



Example
Please see the Bootloader demonstration project's linker control file and startup code for examples.

4.1.2.1 Tasking VX compiler

- ROM images of RAM code are created using the linker's copy directive.
- The linker automatically generates a table which is used to copy all images to all target addresses.
- The startup code includes functionality to copy the images to RAM. This functionality is already included in the example startup codes provided by the compiler.

4.1.2.2 HighTec/GNU compiler

- ROM images of RAM code are created using the linker's "AT >" expression.
- The sections to be copied have to be inserted into a copy table in the linker control file.
- The startup code includes functionality to copy the images to RAM. This functionality is already included in the example startup codes provided by the compiler.
- The compiler option "-ffunction-sections" has to be used. Otherwise, the linker control file cannot address single functions.

5 API Description

5.1 Flash Driver (TC3xx)

The internal flash driver for the Tricore TC3xx supports additional HW specific APIs, which extends the HIS interface defined in [5].

5.1.1 FlashDriver_UcbEraseSync

Prototype	
`IO_ErrorType` **FlashDriver_UcbEraseSync** `(tFblWrapperFlash_UcbBlock block)`	
Parameter	
`block`	Index of UCB block to erase.
Return code	
`IO_E_OK`	Erase of the UCB block was successful.
`<other>`	Erase failed, please refer to the error codes defined in flashdrv.h.
Functional Description	
Erases the given UCB block.	
Particularities and Limitations	
> UCB protection is unlocked (Disable Protection sequence) > FLASH_ENABLE_TRICORE_UCB_ERASE needs to be set	
Call context	
-	

Table 5-1 API Description: FlashDriver_UcbEraseSync

5.1.2 FlashDriver_UcbWriteSync

Prototype	
`IO_ErrorType` **FlashDriver_UcbWriteSync** `(const IO_MemPtrType writeBuffer, IO_SizeType writeLength, tFblWrapperFlash_UcbBlock block, IO_PositionType writeOffset)`	
Parameter	
`writeBuffer`	Pointer to input data buffer.
`writeLength`	Data length. The length has to be aligned to the smallest writeable segment (8 Bytes in data flash).
`block`	Index of UCB block to be written.
`writeOffset`	Write target address offset from begin of UCB block (in flash memory). The target address has to be aligned to the smallest writeable segment (8 Bytes in data flash).
Return code	
`IO_E_OK`	Write of the UCB block was successful.
`<other>`	Write failed, please refer to the error codes defined in flashdrv.h.

Kommentiert [TM1]: There is now way to be more precise on the return value? Is actually a write operation so maybe you can specify a generic Flash write error? This apply also for the other “<other>”

Functional Description
Write data in UCB Block.
Particularities and Limitations
> FLASH_ENABLE_TRICORE_UCB_WRITE needs to be set
Call context
-

Table 5-2 API Description: FlashDriver_UcbWriteSync

5.1.3 FlashDriver_UcbWriteConfirmed

Prototype	
`IO_ErrorType` **FlashDriver_UcbWriteConfirmed** `(tFblWrapperFlash_UcbBlock ucbBlock, tFblWrapperFlash_UcbConfirmation state)`	
Parameter	
`ucbBlock`	Index of UCB block to be written.
`state`	Confirmation state: > FBLFLIO_UCB_CONFIRMATION_UNLOCKED > FBLFLIO_UCB_CONFIRMATION_CONFIRMED
Return code	
`IO_E_OK`	Write of the UCB confirmation was successful.
`<other>`	Write failed, please refer to the error codes defined in flashdrv.h.
Functional Description	
Write CONFIRMATION field in UCB.	
Particularities and Limitations	
> FLASH_ENABLE_TRICORE_UCB_WRITE needs to be set > It is allowed to write CONFIRMED over a UNLOCKED value without erasing the UCB.	
Call context	
-	

Table 5-3 API Description: FlashDriver_UcbWriteConfirmed

5.1.4 FlashDriver_UcbReadSync

Prototype	
`IO_ErrorType` **FlashDriver_UcbReadSync** `(IO_MemPtrType readBuffer, IO_SizeType readLength, tFblWrapperFlash_UcbBlock block, IO_PositionType readOffset)`	
Parameter	
`readBuffer`	Pointer to data buffer.
`readLength`	Data length.
`block`	Index of UCB block to be read.
`readOffset`	Read address offset from begin of UCB block.

Return code	
IO_E_OK	Read of the UCB block was successful.
IO_E_ERASED	Read area is erased
<other>	Write failed, please refer to the error codes defined in flashdrv.h.
Functional Description	
Read data from UCB Block.	
Particularities and Limitations	
> FLASH_ENABLE_TRICORE_UCB_READ needs to be set	
Call context	
-	

Table 5-4 API Description: FlashDriver_UcbReadSync

5.1.5 FlashDriver_UcbDisableProtection

Prototype	
`IO_ErrorType` **FlashDriver_UcbDisableProtection** `(tFblWrapperFlash_UcbBlock block, const vuint32 password[8u])`	
Parameter	
block	Index of UCB block to unlock (ORIG and COPY will be unlocked together).
password	Password for unlock sequence.
Return code	
IO_E_OK	Disabling protection was successful.
IO_E_NOT_OK	Disabling protection failed.
Functional Description	
Execute Disable Protection Sequence.	
Particularities and Limitations	
> COPY and ORIG of UCB block must be in state CONFIRMED > FLASH_ENABLE_TRICORE_UCB_DISABLE_PROTECTION needs to be set.	
Call context	
-	

Table 5-5 API Description: FlashDriver_UcbDisableProtection

5.1.6 FlashDriver_ResumeProtection

Prototype	
`void` **FlashDriver_ResumeProtection** `( void )`	
Parameter	
-	-

Return code	
-	-
Functional Description	
Execute Resume Protection Sequence.	
Particularities and Limitations	
-	
Call context	
-	

Table 5-6 API Description: FlashDriver_ResumeProtection

5.1.7 FlashDriver_DisableProtection

Prototype	
`IO_ErrorType` **FlashDriver_DisableProtection**`(vuint16 uc, const vuint32 password[8u])`	
Parameter	
`uc`	Identification of the UCBx for which the password checking shall be performed.
`password`	Password for unlock sequence.
Return code	
`IO_E_OK`	Disabling protection was successful.
`IO_E_NOT_OK`	Disabling protection failed.
Functional Description	
Execute Disable Protection Sequence.	
Particularities and Limitations	
> COPY and ORIG of UCB block must be in state CONFIRMED	
Call context	
-	

Table 5-7 API Description: FlashDriver_DisableProtection

5.1.8 FlashDriver_ReplaceLogicalSector

Prototype	
`IO_ErrorType FlashDriver_ReplaceLogicalSector( IO_PositionType replaceAddress )`	
Parameter	
`replaceAddress`	Start address of a logical sector which shall be replaced.
`password`	Password for unlock sequence.
Return code	
`IO_E_OK`	Replace of logical sector was successful.

<other>	Replace of logical sector failed, please refer to the error codes defined in flashdrv.h.
Functional Description	
Replaces the logical sector at replaceAddress by a redundant sector	
Particularities and Limitations	
> Flash driver must be initialized. > FLASH_ENABLE_REPLACE_LOGICAL_SECTOR_API needs to be set.	
Call context	
-	

Table 5-8 API Description: FlashDriver_ReplaceLogicalSector

5.1.9 FlashDriver_GetReplaceResult

Prototype	
`vuint32` **FlashDriver_GetReplaceResult** `( void )`	
Parameter	
–	-
Return code	
`replaceResult`	Result of last Replace Sector operation.
Functional Description	
Returns the replaceResult of the last flash operation. If the flash driver detects erroneous flash on write or erase, it will trigger a replace operation (replace the affected logical sector by a redundant sector). For erase operations this is transparent for the caller. GetReplaceResult can be used to detect if the flash driver has replaced any sectors.	
Particularities and Limitations	
> FLASH_ENABLE_REPLACE_LOGICAL_SECTOR_API needs to be set.	
Call context	
-	

Table 5-9 API Description: FlashDriver_GetReplaceResult

5.1.10 FlashDriver_SwapPartitions

Prototype	
`IO_ErrorType` **FlashDriver_SwapPartitions** `( void )`	
Parameter	
–	-
Return code	
`IO_E_OK`	Swapping was successful.
<other>	Swapping failed, please refer to the error codes defined in flashdrv.h.

Functional Description
Swap partitions on the device.
Particularities and Limitations
> Flash driver must be initialized, swap feature must be activated and active. > FLASH_ENABLE_TRICORE_SWAP needs to be set. > FLASH_ENABLE_TRICORE_SWAP_ROUTINE needs to be set.
Call context
-

Table 5-10 API Description: FlashDriver_SwapPartitions

6 Glossary and Abbreviations

6.1 Abbreviations

Abbreviation	Description
BMHD	Boot Mode Header
BMI	Boot Mode Index
CAN	Controller Area Network
ECC	Error Correction Code
ECU	Electronic Control Unit
FBL	Flash Bootloader
NMI	Non-Maskable Interrupt
SFR	Special Function Register
UCB	User Configuration Block
WOP	Write-once Protected Block

© 2020 Vector Informatik GmbH

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2020 Vector Informatik GmbH